

1 – Enunciado: Se cuenta con una lista de 1 millón de números enteros únicos y se necesita comprobar si un número dado está en la colección miles de veces. Eligir una estructura de datos que minimice el tiempo total de las búsquedas.

```
1. def main_ejercicio_1():
2.     numeros = set() #utilizando set para O(n) al crear, O(1) al buscar
3.     for i in range(1_000_000):
4.         numeros.add(i)
5.
6.     print(999_999 in numeros)
7.     print(1_000_001 in numeros)
8.
```

Ejercicio 1:
True
False

2 - Enunciado: Se necesita almacenar datos de 1 millón de usuarios y poder acceder rápidamente al usuario nº 500,000 sin recorrer la colección.

```
1. @dataclass(slots=True) # slots reduce memoria
2. class Usuario:
3.     id: int
4.     nombre: str
5.     edad: int
6.
7. def main_ejercicio_2():
8.     usuarios = [Usuario(i, f"User {i}", 20 + (i % 50)) for i in range(1_000_000)]
9.     u = usuarios[499_999] #acceso por indice es siempre O(1)
10.    print(u)
11.
```

Ejercicio 2:
Usuario(id=499999, nombre='User 499999', edad=69)

3- Enunciado: Se quiere almacenar datos de productos en el orden en el que llegan, pero también poder buscar un producto por su código en $O(1)$.

```
1. @dataclass(slots=True)
2. class Producto:
3.     codigo: int
4.     nombre: str
5.     precio: float
6.
7. #dict mantiene el orden de llegada
8. productos: dict[int, Producto] = {}
9.
10. #insercion en O(1)
11. def alta(prod: Producto) -> None:
12.     productos[prod.codigo] = prod
13.
14. #busqueda en O(1) promedio
15. def buscar(codigo: int) -> Producto | None:
16.     return productos.get(codigo)
17.
18. def iterar_en_orden_llegada():
19.     for p in productos.values():
20.         yield p
21.
22. def main_ejercicio_3():
23.     for i in range(1, 6):
24.         alta(Producto(i, f"Producto {i}", i * 10.0))
25.
26.     print(buscar(3))
27.     print(buscar(10))
28.
29.     print("Productos en orden de llegada:")
30.     for p in iterar_en_orden_llegada():
31.         print(p)
32.
```

```
Ejercicio 3:
Producto(codigo=3, nombre='Producto 3', precio=30.0)
None
Productos en orden de llegada:
Producto(codigo=1, nombre='Producto 1', precio=10.0)
Producto(codigo=2, nombre='Producto 2', precio=20.0)
Producto(codigo=3, nombre='Producto 3', precio=30.0)
Producto(codigo=4, nombre='Producto 4', precio=40.0)
Producto(codigo=5, nombre='Producto 5', precio=50.0)
```

4- Enunciado: Se necesita procesar solicitudes en el orden en que llegan, quitando siempre la primera en $O(1)$.

```
1. cola = deque() #deque permite O(1) en ambos extremos
2.
3. def ingresar_solicitud(solicitud: str) -> None:
4.     cola.append(solicitud)
5.
6. def procesar_solicitud() -> str | None:
7.     if not cola:
8.         return None
9.     return cola.popleft()
10.
11.
12.
13. def main_ejercicio_4():
14.     ingresar_solicitud("Solicitud 1")
15.     ingresar_solicitud("Solicitud 2")
16.     ingresar_solicitud("Solicitud 3")
17.
18.     print(procesar_solicitud())
19.     print(procesar_solicitud())
20.     print(procesar_solicitud())
21.     print(procesar_solicitud())
22.
```

```
Ejercicio 4:
Solicitud 1
Solicitud 2
Solicitud 3
None
```

5- Enunciado: Se tiene una secuencia de eventos y se quiere eliminar duplicados pero manteniendo el orden original.

```
1. eventos = []
2.
3. def agregar_evento(evento: str) -> None:
4.     eventos.append(evento)
5.
6. def eliminar_duplicados(lista) -> None:
7.     vistos = set()
8.     out = []
9.     for x in lista:          # O(n)
10.        if x not in vistos:  # O(1)
11.            vistos.add(x)
12.            out.append(x)
13.     eventos[:] = out
14.
15. def main_ejercicio_5():
16.     agregar_evento("Evento 1")
17.     agregar_evento("Evento 2")
18.     agregar_evento("Evento 1")
19.     agregar_evento("Evento 3")
20.     agregar_evento("Evento 2")
21.
22.     print("Eventos antes de eliminar duplicados:", eventos)
23.     eliminar_duplicados(eventos)
24.     print("Eventos después de eliminar duplicados:", eventos)
25.
```

Ejercicio 5:

Eventos antes de eliminar duplicados: ['Evento 1', 'Evento 2', 'Evento 1', 'Evento 3', 'Evento 2']

Eventos después de eliminar duplicados: ['Evento 1', 'Evento 2', 'Evento 3']